SHADOWINSPECT

A Machine Learning-Driven On-Device Android Forensic Auditing and Threat Intelligence Framework Integrated with MITRE ATT&CK®

Author:	Malik Muhammad Arbab Tahir	**University:**	Air University Multan Campus
Credentials:	Candidate, BS Cyber Security (8th Sem)	**Student ID:**	223613
Repository:	github.com/arbab-tahir/ShadowInspect	**Platform:**	Android (Kotlin/Compose)

Executive Summary

ShadowInspect is an advanced, production-ready Android mobile application built to democratize local security auditing, binary static analysis, forensic data analysis, and real-time threat intelligence. Engineered with a modular clean architecture using MVVM, Hilt, Room, and Retrofit, the application enables system administrators, penetration testers, and security professionals to conduct deep local APK binary evaluations, scan URLs against unified multi-engine APIs, and map discovered behaviors directly to the global **MITRE ATT&CK** knowledge base.

1 Key Architectural & Engineering Pillars

- **Jetpack Compose UI Framework:** Constructed on a reactive, cyber-neon themed visual layout with zero xml dependency, incorporating a typewriter terminal-reveal animation engine and customizable dark mode rendering context.
- **Modular Dependency Injection:** Leverages **Dagger-Hilt** to control scoping and enforce low coupling across repository layers, database modules, and remote threat engines.
- **Offline Binary Analyzer:** Built with embedded local machine learning parsing mechanisms (**TensorFlow Lite**) to classify APK executable file risks on-device without exposing sensitive file footprints to network structures.
- **Persistent Datastore caching:** Employs a robust offline-first caching policy powered by **Room DB** and **Kotlin Flow/StateFlow** to ensure seamless access to historical scan records and local MITRE matrices.

2 Core Functional Modules

2.1 1. On-Device Static & Forensic Analysis

Performs safe, local static analysis on target application APK files:

- Parses package configurations, active permissions, and structural intent flags on-device.
- Runs features through a local TensorFlow Lite deep learning model to flag statistical risk vectors.
- Extracts metadata (EXIF tags) and structures from documents (PDFs, DOCX) to find macros, tracking links, and embedded exploits.

2.2 2. MITRE ATT&CK Mobile Matrix Integration

Maps local app behavior patterns directly to standardized adversarial tactics:

- Implements full offline browsing of the Mobile MITRE ATT&CK framework dataset (49 MB JSON footprint cached locally).
- Maps suspicious manifest entries (e.g. system broadcast capture, device admin requests) to corresponding MITRE Techniques.
- Provides security engineers with standardized threat IDs and verified defensive mitigation advice immediately.

2.3 3. Unified Threat Intelligence Cloud Engine (API Orchestrator)

Consolidates external scans into a single action-flow:

- Combines multi-engine domain evaluations using VirusTotal, URLScan, IPQualityScore, and Veriphone APIs.
- Handles environment-based secure loading of client credentials dynamically, preventing keys from leaking to repository logs.

2.4 4. Interactive Security Education System

Promotes active learning for security analysts:

- Hosts structured, offline modules outlining secure coding practices, cryptography basics, and reverse engineering defenses.
- Tracks learning progress locally within SQLite/Room databases to Gamify mobile security awareness.

3 System Specifications & Engineering Stack

Component	Technology Implemented
Programming Language	Kotlin (v1.9.x), Coroutines, Flow, StateFlow
UI Engine	Jetpack Compose, Material Design 3, Lottie, custom canvas rendering
Data Persistence	Room SQLite Database, Proto DataStore
Network Core	Retrofit 2, OkHttp 4, Gson serialization
Machine Learning	TensorFlow Lite Interpreter (Interpreter API v2.14.0)
Dependency Injection	Dagger-Hilt (HiltAndroidApp, @Inject, @Provides)
Security Infrastructure	Git LFS for binary datasets, local.properties abstraction for API keys

4 Security & Repository Hardening

Prior to repository deployment, the codebase underwent standard security hardening to protect intellectual property and developer workspace metadata:

- Hardcoded local paths (e.g., machine-specific JDK targets and custom compiler build outputs) were extracted.
- Sensitive client secrets (API keys) were abstracted into dynamic gradle properties via local configuration templates (‘local.properties.example‘).
- Heavy binary files (TFLite weights and comprehensive MITRE models) were configured under Git Large File Storage (LFS) to ensure repository execution speed and health.